

Laboratory 8

TASK: MULTIPROCESSING SORTING USING PIPES

Students will implement a **three-process sorting system** where processes communicate using unidirectional pipes and non-blocking receive.

System Description

- Process 1:
 - Reads 220 numbers from a file.
 - Splits them into two arrays.
 - Sends each array to worker processes via pipes.
- Process 2: Sorts the first array.
- Process 3: Sorts the second array.
- After both workers finish, Process 1 merges the sorted arrays.

Tasks:

1. Implement three processes (process 1 + two worker processes).
 - Use C++ (linux) or Python (windows) multiprocessing. (1 point)
2. Use pipes for communication.
 - Pipes are unidirectional, so use two pipes per communication direction if sending and receiving.
 - Ensure correct parent-child pipe structure. (1 point)
3. Implement non-blocking receive using poll().
 - Check for incoming data every 0.2 seconds.
 - Avoid blocking worker processes. (1 point)
4. Sort the arrays in Process 2 and Process 3.
 - Use any sorting algorithm (built-in or custom). (0.75 point)
5. Wait for worker processes to finish before continuing.
 - Use join() (Python) or waitpid() (C++). (0.25 point)
6. Merge the two sorted arrays using Process 1. (0.5 point)
7. Measure the execution time for the sorting and print it. (0.5 point)

NOTE: Upload the print-screen of final code execution on enauczanie and of course your C++/Python code :)